SLAM : Projet Agora Sprint 5, demande 4

Mise en place d'une relation many to many avec le composant Doctrine

Auteur : *Yannick NARBEY*

Pré-requis :

- Symfony : contrôleurs, routes, templates twig
- Agora Sprint 5 demande 3 : création formulaire CRUD tournoi avec Doctrine

TABLE DES MATIERES

1. Contexte et objectifs ***2***

2. Réalisation ***3***

2.1 Création de l'entité Participant **3**

2.2 Création de la relation entre les entités Participant et Tournoi **4**

2.2.1 Ajout d'une relation many to many 4

2.2.2 Contrôler le code généré 5

2.3 Création de la table participant par migration **11**

2.3.1 Générer la migration 11

2.3.2 Appliquer la migration 11

2.4 Génération des CRUD **13**

2.4.1 Génération des CRUD de gestion des tournois 13

2.4.2 Mise en œuvre de l'association many to many dans les 2 formulaires 14

2.4.3 Tests 16

2.4.4 Convection du type de l'objet :Tournoi 17

2.4.5 Ajout du nouveau formulaire au menu 20

2.4.6 Charter et franciser le nouveau formulaire 22

2.4.7 Améliorations fonctionnelles 23

3. Conclusion ***25***

4. A vous de jouer ***25***

1. Contexte et objectifs

Le directeur de l'association Agora demande à pouvoir gérer l'inscription de participants aux tournois.

L'ESN Logma est toujours en charge de mener les évolutions sur l'application Agora.

Dans cette itération, vous êtes chargé de mettre en œuvre ces nouvelles fonctionnalité à l'aide du composant Doctrine et d'une relation many to many ainsi que les fixtures (les données de tests).

Symfony (rappel)

Symfony est un ensemble de composants PHP, un **framework d'application Web**, une philosophie et une communauté - tous travaillant ensemble en harmonie. (site officiel)
Symfony est un framework basé sur le modèle MVC qui permet de réaliser des sites complexes rapidement, mais de façon structurée et avec un code clair et maintenable.
SensioLabs est le créateur du framework PHP Symfony. Le nom SensioLabs vient du mot espagnol "sencillo" qui signifie "simple". Fabien Potencier est le fondateur et chef de projet Symfony.

https://symfony.com/

Doctrine (rappel)

Symfony est livré par défaut avec l'**ORM** Doctrine. Un ORM (pour **Object-Relation Mapper**, « lien objet-relation ») est un programme qui se place en interface entre un programme applicatif et une base de données relationnelle pour simuler une base de données orientée objet. Il définit des **correspondances** entre les **schémas de la base** de données et les **classes** du programme applicatif.

https://www.doctrine-project.org

Entité (rappel)

Les **données** de la base de données sont des **objets**. Un objet dont l'enregistrement est confié à l'ORM se nomme une **entité** (entity en anglais). On dit également **persister une entité**, plutôt qu'enregistrer une entité.

Attributs PHP et Doctrine (rappel)

Doctrine utilise les attributs PHP 8 afin de définir la table correspondant à une entité, définir un identifiant (id) en auto-incrément, nommer les colonnes, etc. Ces informations se nomment les metadata de l'entité.
Ajouter les metadata à un objet, s'appelle **mapper l'objet, c'est**-à-dire faire le lien entre l'objet et la représentation physique qu'utilise Doctrine (une table de la base de données relationnelle).

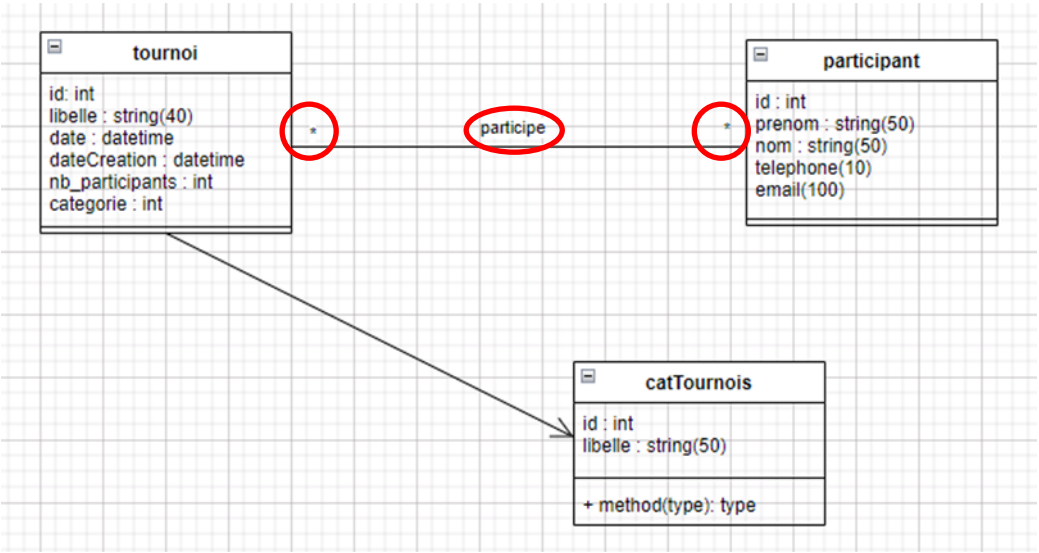
Le développement de l'application se fera dans un cadre de travail agile **SCRUM**.

2. Réalisation

Afin de d'organiser des tournois ouverts à des participants extérieurs à l'association, il faut ajouter une entité participant.

Un participant pourra participer à 1 ou plusieurs tournois et naturellement un tournoi accueillera plusieurs participants.

Pour Doctrine ce cas correspond au **type d'association** : **ManyToMany**



2.1 Création de l'entité Participant

Créer l'entité participant avec la commande make:entity :

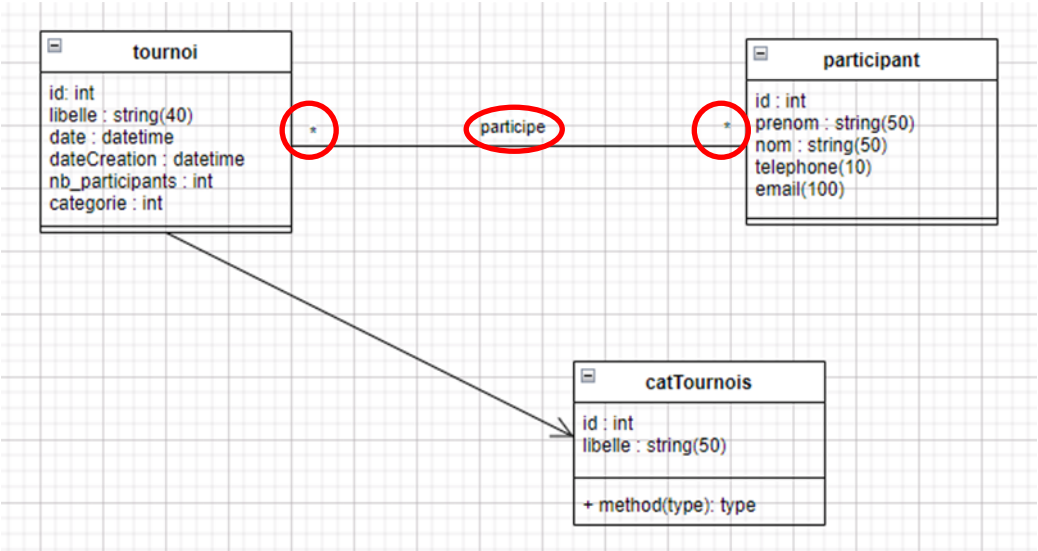
```
C:\wamp64\www\agoram62>php bin/console make:entity Participant
```

```
C:\Windows\System32\cmd.e
C:\wamp64\www\agoram54>php bin/console make:entity Participant

 created: src/Entity/Participant.php
 created: src/Repository/ParticipantRepository.php

 Entity generated! Now let's add some fields!
 You can always add more fields later manually or by re-running this command.

 New property name (press <return> to stop adding fields):
 > prenom

 Field type (enter ? to see all types) [string]:
 >

 Field length [255]:
 > 50

 Can this field be null in the database (nullable) (yes/no) [no]:
 >

 updated: src/Entity/Participant.php

 Add another property? Enter the property name (or press <return> to stop adding fields):
 > nom

 Field type (enter ? to see all types) [string]:
 >

 Field length [255]:
 > 50
```

```
Can this field be null in the database (nullable) (yes/no) [no]:
>

updated: src/Entity/Participant.php

Add another property? Enter the property name (or press <return> to stop adding fields):
> telephone

Field type (enter ? to see all types) [string]:
>

Field length [255]:
> 14

Can this field be null in the database (nullable) (yes/no) [no]:
>

updated: src/Entity/Participant.php

Add another property? Enter the property name (or press <return> to stop adding fields):
> email

Field type (enter ? to see all types) [string]:
>

Field length [255]:
> 100

Can this field be null in the database (nullable) (yes/no) [no]:
> yes

updated: src/Entity/Participant.php

Add another property? Enter the property name (or press <return> to stop adding fields):
>


Success!

Next: When you're ready, create a migration with php bin/console make:migration
```

2.2 Création de la relation entre les entités Participant et Tournoi

Pour rappel, un participant pourra participer à 1 ou plusieurs tournois et naturellement un tournoi accueillera plusieurs participants.



Pour Doctrine ce cas correspond au **type d'association** : **ManyToMany**

2.2.1 Ajout d'une relation many to many

Il faut créer la relation Many To Many entre les entités Participant et Tournoi, en utilisant make:entity, commencont pas modifier Tournoi :

```
C:\wamp64\www\agoram54>php bin/console make:entity Tournoi
```

```
C:\Windows\System32\cmd.e

C:\wamp64\www\agoram54>php bin/console make:entity Tournoi

 Your entity already exists! So let's add some new fields!

 New property name (press <return> to stop adding fields):
 > participants

 Field type (enter ? to see all types) [string]:
 > relation

 What class should this entity be related to?:
 > Participant

What type of relationship is this?
 ------------ ---------------------------------------------------------------------------
  Type         Description
 ------------ ---------------------------------------------------------------------------
  ManyToOne    Each Tournoi relates to (has) one Participant.
               Each Participant can relate to (can have) many Tournoi objects.

  OneToMany    Each Tournoi can relate to (can have) many Participant objects.
               Each Participant relates to (has) one Tournoi.

  ManyToMany   Each Tournoi can relate to (can have) many Participant objects.
               Each Participant can also relate to (can also have) many Tournoi objects.

  OneToOne     Each Tournoi relates to (has) exactly one Participant.
               Each Participant also relates to (has) exactly one Tournoi.
 ------------ ---------------------------------------------------------------------------
```

```
 Relation type? [ManyToOne, OneToMany, ManyToMany, OneToOne]:
 > ManyToMany

 Do you want to add a new property to Participant so that you can access/update Tournoi objects from it - e.g. $part
icipant->getTournois()? (yes/no) [yes]:
 >

 A new property will also be added to the Participant class so that you can access the related Tournoi objects from
it.

 New field name inside Participant [tournois]:
 >

 updated: src/Entity/Tournoi.php
 updated: src/Entity/Participant.php

 Add another property? Enter the property name (or press <return> to stop adding fields):
 >


 Success!


 Next: When you're ready, create a migration with php bin/console make:migration
```

2.2.2 Contrôler le code généré

Dans le fichier entity/Tournoi.php :

- une collection de *participants* a été ajoutée avec son getter `getParticipants()` qui retourne une collection
- et 2 méthodes pour ajouter (`addParticipant()`) et supprimer (`removeParticipant()`) un participant.

Fichier agoram54/src/Entity/Tournoi.php

```php
<?php

namespace App\Entity;

use App\Repository\TournoiRepository;
```

```php
use Doctrine\Common\Collections\ArrayCollection;
use Doctrine\Common\Collections\Collection;
use Doctrine\DBAL\Types\Types;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: TournoiRepository::class)]
class Tournoi
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 40)]
    private ?string $libelle = null;

    #[ORM\Column(type: Types::DATETIME_MUTABLE)]
    private ?\DateTimeInterface $date = null;

    #[ORM\Column(type: Types::DATETIME_MUTABLE)]
    private ?\DateTimeInterface $dateCreation = null;

    #[ORM\ManyToOne(inversedBy: 'tournois')]
    private ?CatTournois $categorie = null;

    #[ORM\Column(nullable: true)]
    private ?int $nb_participants = null;

    /**
     * @var Collection<int, Participant>
     */
    #[ORM\ManyToMany(targetEntity: Participant::class, inversedBy: 'tournois')]
    private Collection $participants;

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getLibelle(): ?string
    {
        return $this->libelle;
    }

    public function setLibelle(string $libelle): static
    {
        $this->libelle = $libelle;

        return $this;
    }

    public function getDate(): ?\DateTimeInterface
    {
```

```
        return $this->date;
    }

    public function setDate(\DateTimeInterface $date): static
    {
        $this->date = $date;

        return $this;
    }

    public function getDateCreation(): ?\DateTimeInterface
    {
        return $this->dateCreation;
    }

    public function setDateCreation(\DateTimeInterface $dateCreation): static
    {
        $this->dateCreation = $dateCreation;

        return $this;
    }

    public function __construct()
    {
        $this->dateCreation = new \DateTime('now');
        $this->participants = new ArrayCollection();
    }

    public function getCategorie(): ?CatTournois
    {
        return $this->categorie;
    }

    public function setCategorie(?CatTournois $categorie): static
    {
        $this->categorie = $categorie;

        return $this;
    }

    public function getNbParticipants(): ?int
    {
        return $this->nb_participants;
    }

    public function setNbParticipants(?int $nb_participants): static
    {
        $this->nb_participants = $nb_participants;

        return $this;
    }

    /**
```

```
     * @return Collection<int, Participant>
     */
    public function getParticipants(): Collection
    {
        return $this->participants;
    }

    public function addParticipant(Participant $participant): static
    {
        if (!$this->participants->contains($participant)) {
            $this->participants->add($participant);
        }

        return $this;
    }

    public function removeParticipant(Participant $participant): static
    {
        $this->participants->removeElement($participant);

        return $this;
    }
}
```

Dans le fichier entity/Participant.php :

- a été créé avec un id, un nom, un prénom, un téléphone, un email et une collection de tournois (attribut tournois)
- ainsi que 3 méthodes : `getTournois(), addTournoi() et removeTournoi().`

Fichier agoram54/src/Entity/Participant.php

```
<?php

namespace App\Entity;

use App\Repository\ParticipantRepository;
use Doctrine\Common\Collections\ArrayCollection;
use Doctrine\Common\Collections\Collection;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: ParticipantRepository::class)]
class Participant
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 50)]
    private ?string $prenom = null;

    #[ORM\Column(length: 50)]
    private ?string $nom = null;
```

```php
    #[ORM\Column(length: 14)]
    private ?string $telephone = null;

    #[ORM\Column(length: 100)]
    private ?string $email = null;

    /**
     * @var Collection<int, Tournoi>
     */
    #[ORM\ManyToMany(targetEntity: Tournoi::class, mappedBy: 'participants')]
    private Collection $tournois;

    public function __construct()
    {
        $this->tournois = new ArrayCollection();
    }

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getPrenom(): ?string
    {
        return $this->prenom;
    }

    public function setPrenom(string $prenom): static
    {
        $this->prenom = $prenom;

        return $this;
    }

    public function getNom(): ?string
    {
        return $this->nom;
    }

    public function setNom(string $nom): static
    {
        $this->nom = $nom;

        return $this;
    }

    public function getTelephone(): ?string
    {
        return $this->telephone;
    }

    public function setTelephone(string $telephone): static
    {
```

```php
        $this->telephone = $telephone;

        return $this;
    }

    public function getEmail(): ?string
    {
        return $this->email;
    }

    public function setEmail(string $email): static
    {
        $this->email = $email;

        return $this;
    }

    /**
     * @return Collection<int, Tournoi>
     */
    public function getTournois(): Collection
    {
        return $this->tournois;
    }

    public function addTournoi(Tournoi $tournoi): static
    {
        if (!$this->tournois->contains($tournoi)) {
            $this->tournois->add($tournoi);
            $tournoi->addParticipant($this);
        }

        return $this;
    }

    public function removeTournoi(Tournoi $tournoi): static
    {
        if ($this->tournois->removeElement($tournoi)) {
            $tournoi->removeParticipant($this);
        }

        return $this;
    }
}
```

Les attributs *mappedBy* et *inversedBy*

Il y a toujours une entité qui est "propriétaire" de la relation. Dans le cas présent, on peut considérer que c'est le tournoi qui est propriétaire de la relation; c'est lors de la création d'un tournoi que les participants seront renseignés.

- L'entité **propriétaire** doit déclarer l'attribut "inversedBy" : il correspond à la **propriété de l'objet** "possédé" qui fait le lien entre les deux entités.
- L'entité **possédée** (ou inverse) doit déclarer l'attribut "mappedBy" : il correspond à la **propriété de l'objet** "propriétaire" qui fait le lien entre les deux entités.

Classe Tournoi :

```
    #[ORM\ManyToMany(targetEntity: Participant::class, inversedBy: 'tournois')]
    private Collection $particpants;
```

Classe Participant :

```
    #[ORM\ManyToMany(targetEntity: Tournoi::class, mappedBy: 'particpants')]
    private Collection $tournois;
```

L'entité propriétaire d'une association bidirectionnelle est celle que Doctrine considère quand il faut déterminer s'il y a une mise à jour à faire au niveau de la base de données.

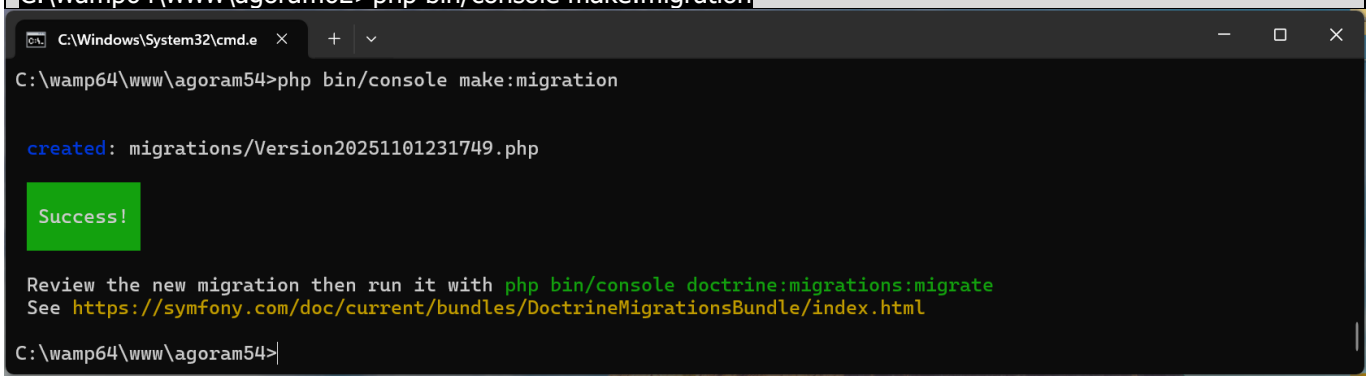
https://www.doctrine-project.org/projects/doctrine-orm/en/latest/reference/unitofwork-associations.html

2.3 Création de la table participant par migration

2.3.1 Générer la migration

On génère les migrations, avec make:migration

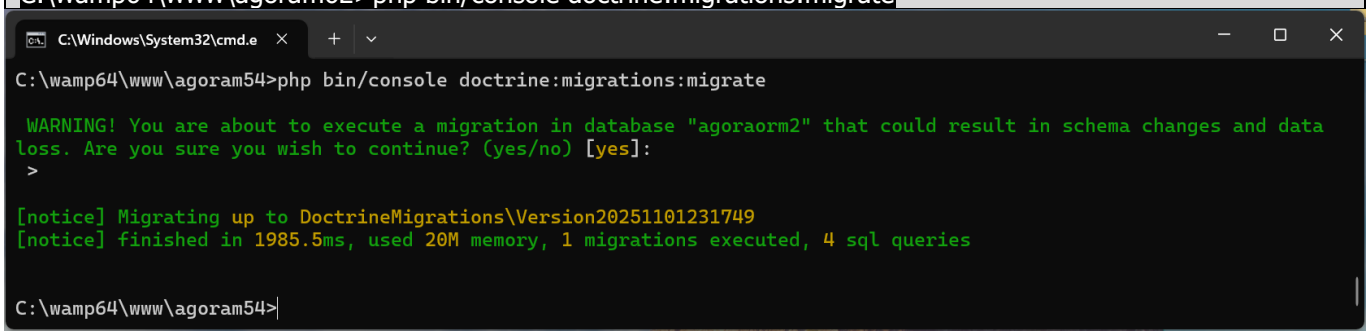
```
C:\wamp64\www\agoram62>php bin/console make:migration
```



2.3.2 Appliquer la migration

Exécuter la migration avec la commande doctrine:migrations:migrate

```
C:\wamp64\www\agoram62>php bin/console doctrine:migrations:migrate
```



Vérifier que la table participant a été créée.

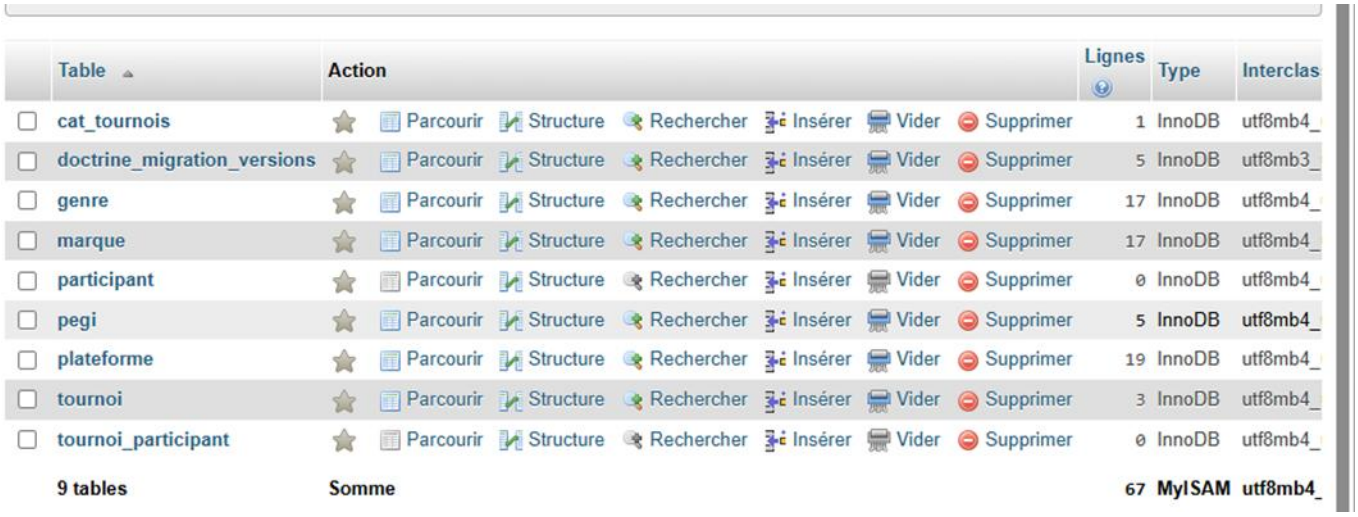
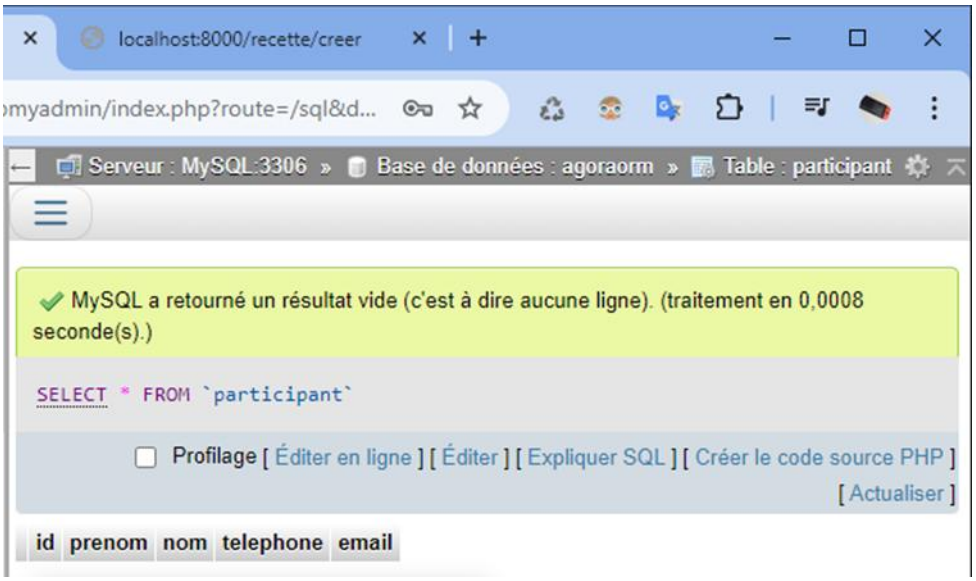
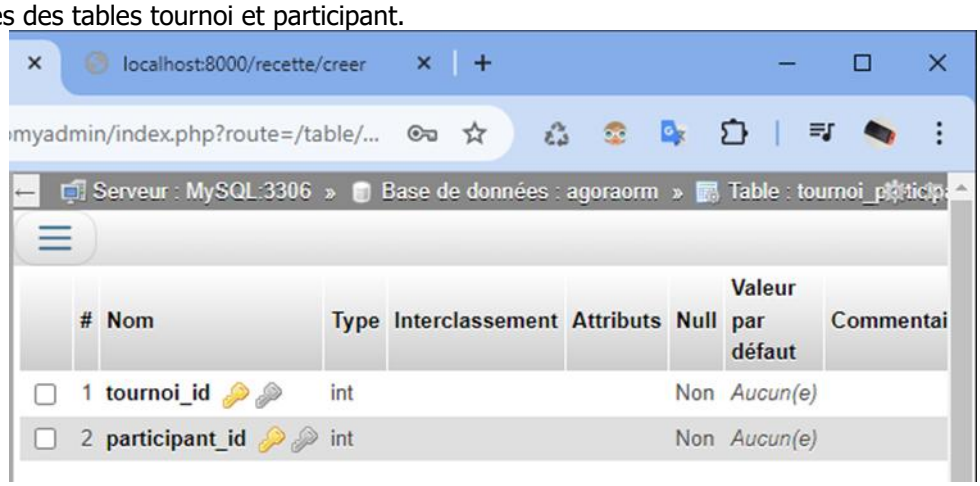


Table	Action	Lignes	Type	Interclas
cat_tournois	Parcourir Structure Rechercher Insérer Vider Supprimer	1	InnoDB	utf8mb4_
doctrine_migration_versions	Parcourir Structure Rechercher Insérer Vider Supprimer	5	InnoDB	utf8mb3_
genre	Parcourir Structure Rechercher Insérer Vider Supprimer	17	InnoDB	utf8mb4_
marque	Parcourir Structure Rechercher Insérer Vider Supprimer	17	InnoDB	utf8mb4_
participant	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_
pegi	Parcourir Structure Rechercher Insérer Vider Supprimer	5	InnoDB	utf8mb4_
plateforme	Parcourir Structure Rechercher Insérer Vider Supprimer	19	InnoDB	utf8mb4_
tournoi	Parcourir Structure Rechercher Insérer Vider Supprimer	3	InnoDB	utf8mb4_
tournoi_participant	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_
9 tables	Somme	67	MyISAM	utf8mb4_

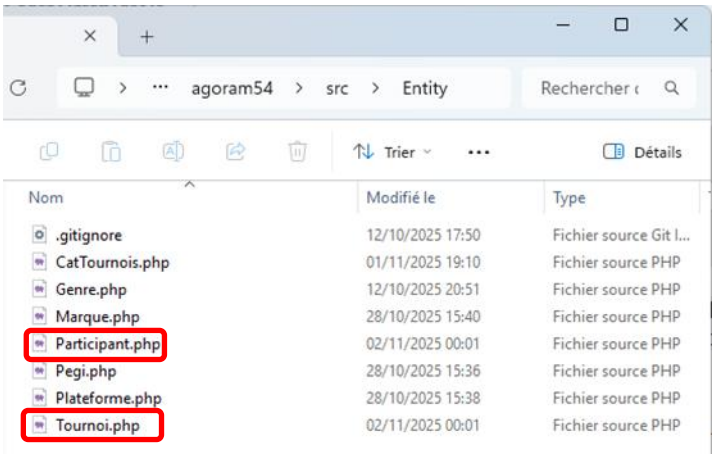


Vérifier qu'une table tournoi_participant a été automatiquement créée avec 2 clés étrangères, qui sont naturellement les clés primaires des tables tournoi et participant.



Actions	Propriétés de la contrainte	Colonne	Contrainte de clé étrangère (INNODB)		
			Base de données	Table	Colonne
Supprimer	FK_9C5314799D1C3019 ON DELETE CASCADE ON UPDATE RESTRICT	participant_id + Ajouter une colonne	agoraorm2	participant	id
Supprimer	FK_9C531479F607770A ON DELETE CASCADE ON UPDATE RESTRICT	tournoi_id + Ajouter une colonne	agoraorm2	tournoi	id

Il faut noter que la représentation de la table **tournoi_participant** est effectuée par le biais des classes Tournoi ($tournoi) et Particpant ($participant) et qu'il n'y a pas d'entité spécifique correspondant à cette table.



2.4 Génération des CRUD

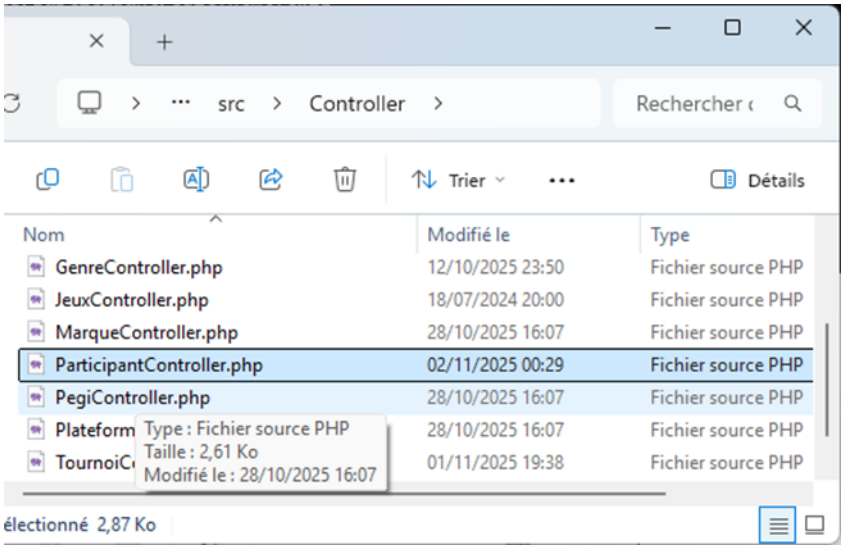
2.4.1 Génération des CRUD de gestion des tournois

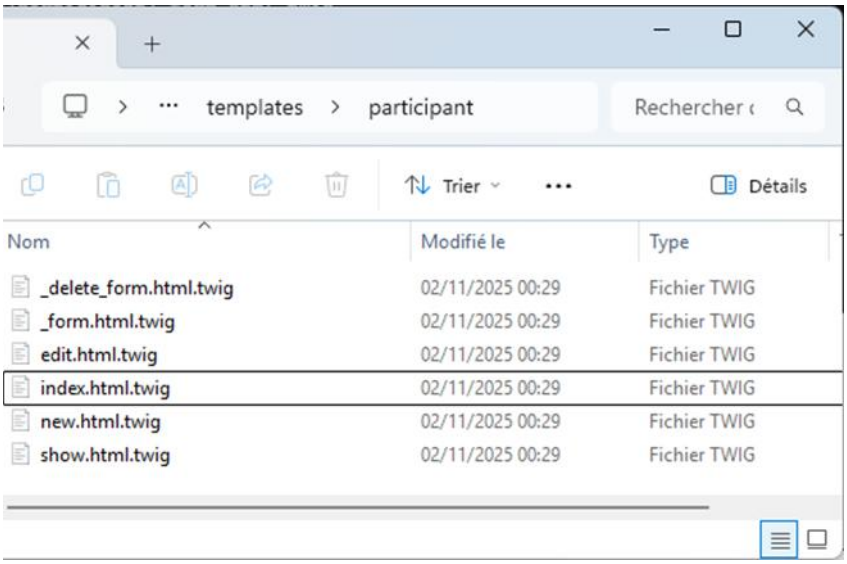
Utiliser la commande make:crud

C:\wamp64\www\agoram54>php bin/console make:crud Participant

```
C:\Windows\System32\cmd.e
C:\wamp64\www\agoram54>php bin/console make:crud Participant

 Choose a name for your controller class (e.g. ParticipantController) [ParticipantController]:
 >

 Do you want to generate tests for the controller?. [Experimental] (yes/no) [no]:
 >

 created: src/Controller/ParticipantController.php
 created: src/Form/ParticipantType.php
 created: templates/participant/_delete_form.html.twig
 created: templates/participant/_form.html.twig
 created: templates/participant/edit.html.twig
 created: templates/participant/index.html.twig
 created: templates/participant/new.html.twig
 created: templates/participant/show.html.twig

 Success!

 Next: Check your new CRUD by going to /participant/

C:\wamp64\www\agoram54>
```

Vérifier la génération des fichiers mentionnés ci-dessus.





2.4.2 Mise en œuvre de l'association many to many dans les 2 formulaires

Le CRUD généré par make:crud ne sait pas gérer automatiquement les relations ManyToMany à moins que d'ajouter le champ correctement dans le FormType et Doctrine ne met pas à jour les relations ManyToMany simplement avec un flush().

Modifier le formulaire type du tournoi src/Form/TournoiType.php et modifier la méthode buildForm() :

Fichier agoram54/src/Form/TournoiType.php

```php
<?php

namespace App\Form;

use App\Entity\Tournoi;
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;
use App\Entity\Participant;
use Symfony\Bridge\Doctrine\Form\Type\EntityType;

class TournoiType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
```

```
    {
        $builder
            ->add('libelle')
            ->add('date')
            ->add('dateCreation')
            ->add('categorie')
            ->add('participants', EntityType::class, [
                'class' => Participant::class,
                'choice_label' => fn (Participant $p) => $p->getPrenom() . '' . $p->getNom(),
                'multiple' => true,
                'expanded' => false,
                'by_reference' => false,
            ])
        ;
    }

    public function configureOptions(OptionsResolver $resolver): void
    {
        $resolver->setDefaults([
            'data_class' => Tournoi::class,
        ]);
    }
}
```

Modifier le formulaire type des participants src/Form/Participant.php et modifier la méthode buildForm() :

Fichier agoram54/src/Form/ParticipantType.php

```
<?php

namespace App\Form;

use App\Entity\Participant;
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;
use App\Entity\Tournoi;
use Symfony\Bridge\Doctrine\Form\Type\EntityType;

class ParticipantType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            ->add('prenom')
            ->add('nom')
            ->add('telephone')
            ->add('email')
            ->add('tournois')
            ->add('tournois', EntityType::class, [
                'class' => Tournoi::class,
                'choice_label' => 'libelle',
                'multiple' => true,
```

```
                'expanded' => false,
                'by_reference' => false,
            ])
        ;
    }

    public function configureOptions(OptionsResolver $resolver): void
    {
        $resolver->setDefaults([
            'data_class' => Participant::class,
        ]);
    }
}
```

2.4.3 Tests

Vider le cache, pour forcer la prise en compte des modifications (nouvelles routes, nouveaux contrôleurs …)

.

```
C:\wamp64\www\agoram54>php bin/console cache:clear
```

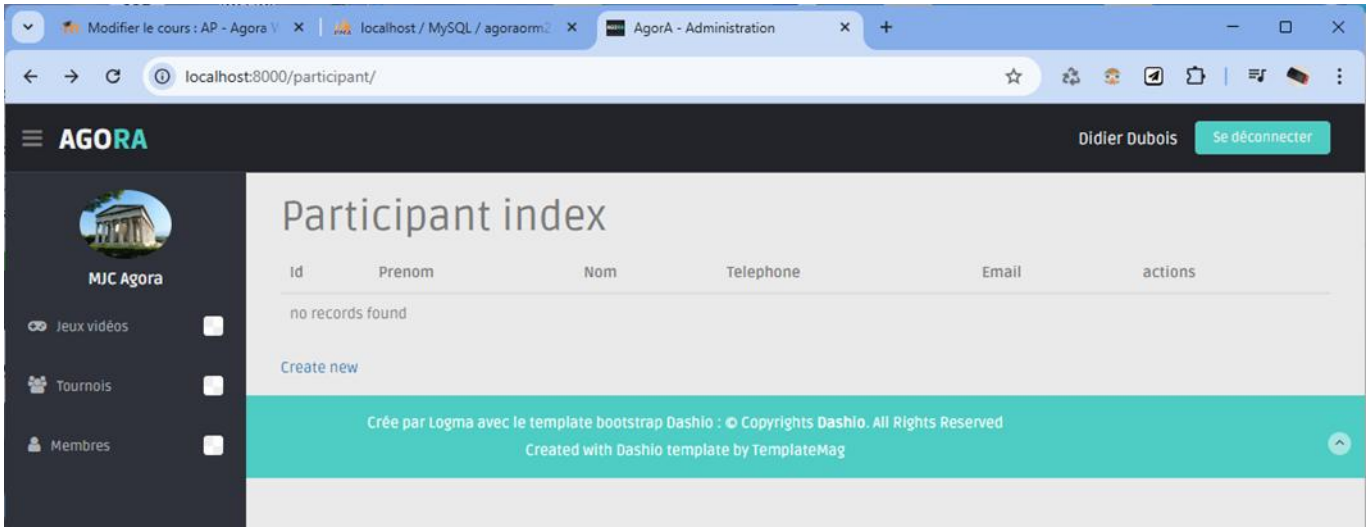
```
C:\Windows\System32\cmd.e
C:\wamp64\www\agoram54>php bin/console cache:clear

 // Clearing the cache for the dev environment with debug true

 [OK] Cache for the "dev" environment (debug=true) was successfully cleared.


C:\wamp64\www\agoram54>
```

Demander la liste des routes et retrouver les routes des nouveaux formulaires app_participant:

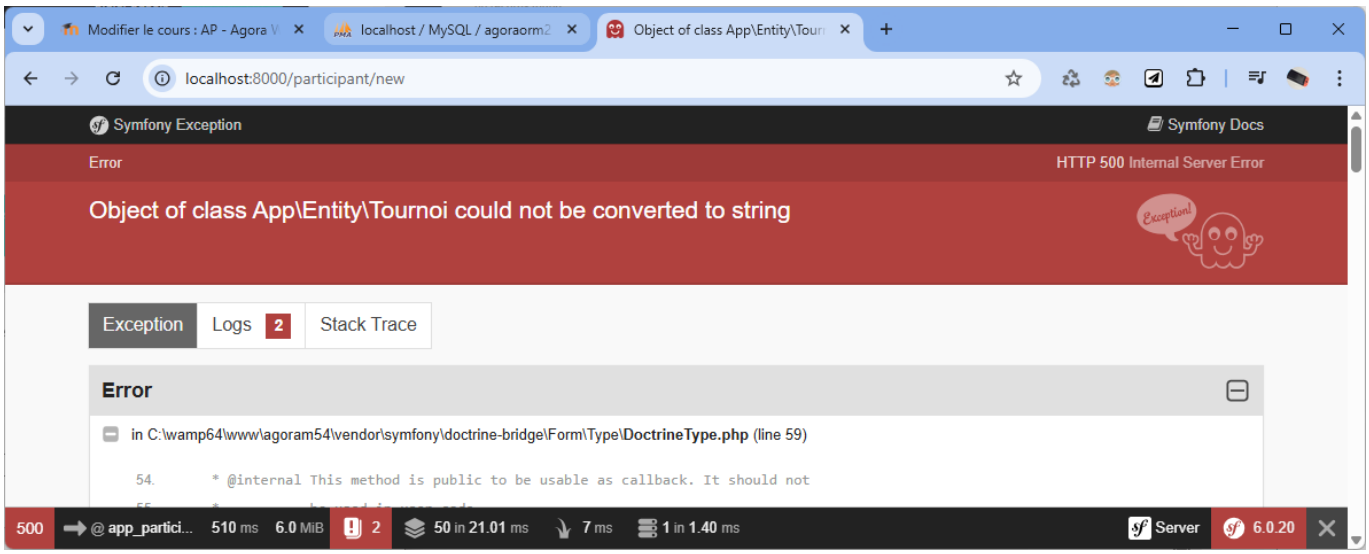
```
C:\Windows\System32\cmd.e
  app_marque_show            GET        ANY      ANY    /marque/{id}
  app_marque_edit            GET|POST   ANY      ANY    /marque/{id}/edit
  app_marque_delete          POST       ANY      ANY    /marque/{id}
  app_participant_index      GET        ANY      ANY    /participant/
  app_participant_new        GET|POST   ANY      ANY    /participant/new
  app_participant_show       GET        ANY      ANY    /participant/{id}
  app_participant_edit       GET|POST   ANY      ANY    /participant/{id}/edit
  app_participant_delete     POST       ANY      ANY    /participant/{id}
  app_pegi_index             GET        ANY      ANY    /pegi/
  app_pegi_new               GET|POST   ANY      ANY    /pegi/new
  app_pegi_show              GET        ANY      ANY    /pegi/{id}
  app_pegi_edit              GET|POST   ANY      ANY    /pegi/{id}/edit
  app_pegi_delete            POST       ANY      ANY    /pegi/{id}
  app_plateforme_index       GET        ANY      ANY    /plateforme/
  app_plateforme_new         GET|POST   ANY      ANY    /plateforme/new
  app_plateforme_show        GET        ANY      ANY    /plateforme/{id}
  app_plateforme_edit        GET|POST   ANY      ANY    /plateforme/{id}/edit
  app_plateforme_delete      POST       ANY      ANY    /plateforme/{id}
  app_tournoi_index          GET        ANY      ANY    /tournoi/
  app_tournoi_new            GET|POST   ANY      ANY    /tournoi/new
  app_tournoi_show           GET        ANY      ANY    /tournoi/{id}
  app_tournoi_edit           GET|POST   ANY      ANY    /tournoi/{id}/edit
  app_tournoi_delete         POST       ANY      ANY    /tournoi/{id}
  app_tournois               ANY        ANY      ANY    /tournois
 ------------------------- ---------- -------- ------ ----------------------------------
```

Lancer le serveur Symfony et tester les nouveaux CRUD /participant

en saisissant l'URL : http://localhost:8000/participant



L'ouverture du formulaire de création des participants provoque une erreur relative aux types de catégories de tournois :



2.4.4 Convection du type de l'objet :Tournoi

Il faut alors ouvrir l'entité Tournois et ajouter une méthode qui permet de convertir l'objet en string comme le souhaite le formulaire de création :

Fichier agora54/src/Entity/Tournois.php

```php
<?php

namespace App\Entity;

use App\Repository\TournoiRepository;
use Doctrine\Common\Collections\ArrayCollection;
use Doctrine\Common\Collections\Collection;
use Doctrine\DBAL\Types\Types;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: TournoiRepository::class)]
class Tournoi
{
    ...
```

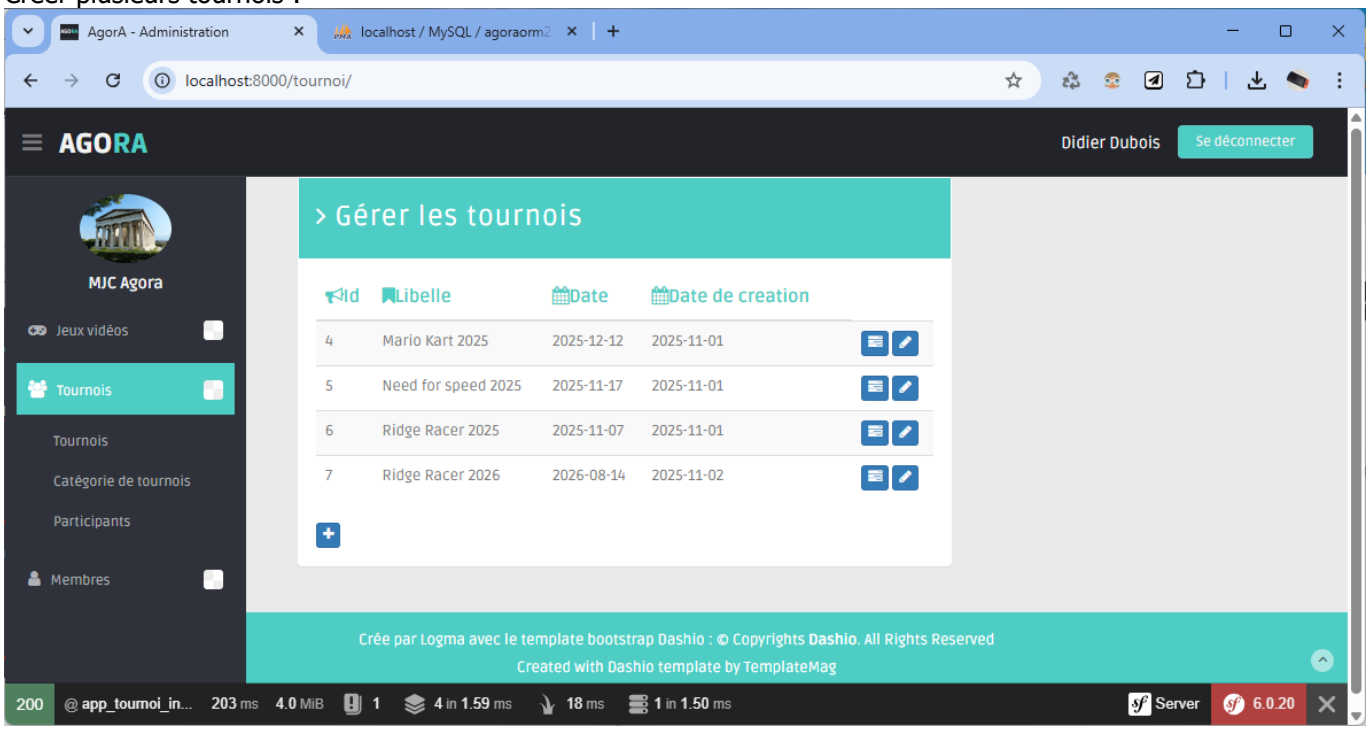
```php
    public function removeParticipant(Participant $participant): static
    {
        $this->participants->removeElement($participant);

        return $this;
    }

    public function __toString(): string
    {
        return $this->libelle;
    }
}
```

Lorsque Symfony construit le formulaire, il génère des lignes HTML avec PHP du type :
echo "<option value='1'>" . (string)$Tournois . "</option>" ;

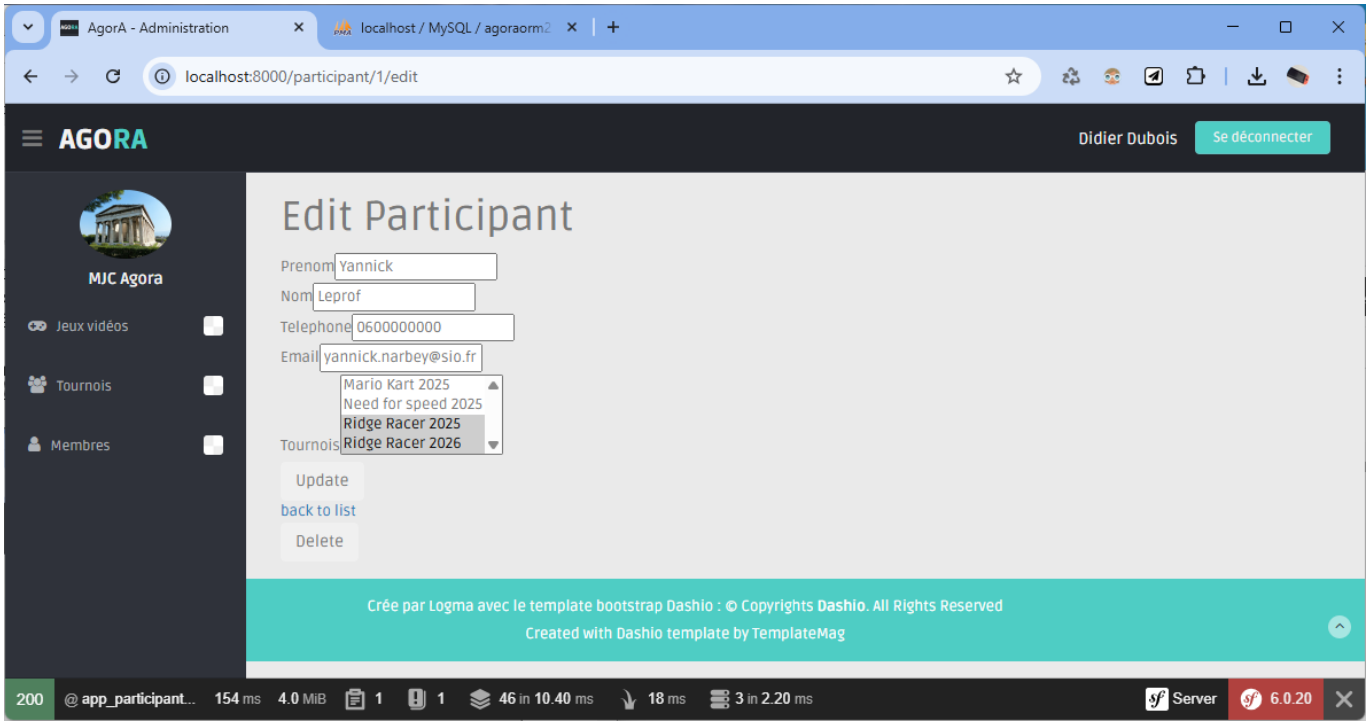
C'est ce cast « string » sur $Tournois qui ne passe pas, car rien n'est prévu pour convertir cet objet en string.
En ajoutant cette méthode public function __toString(): string { return $this->libelle; }, ça renvoie le champ libelle de l'objet Tournois quand un cast « string » est demandé.

Vous pouvez de nouveau créer des participants à inscrire aux tournoi, en saisissant l'URL : http://localhost:8000/participant. La liste des Tournois est correctement remplie avec des (string)$tournois :

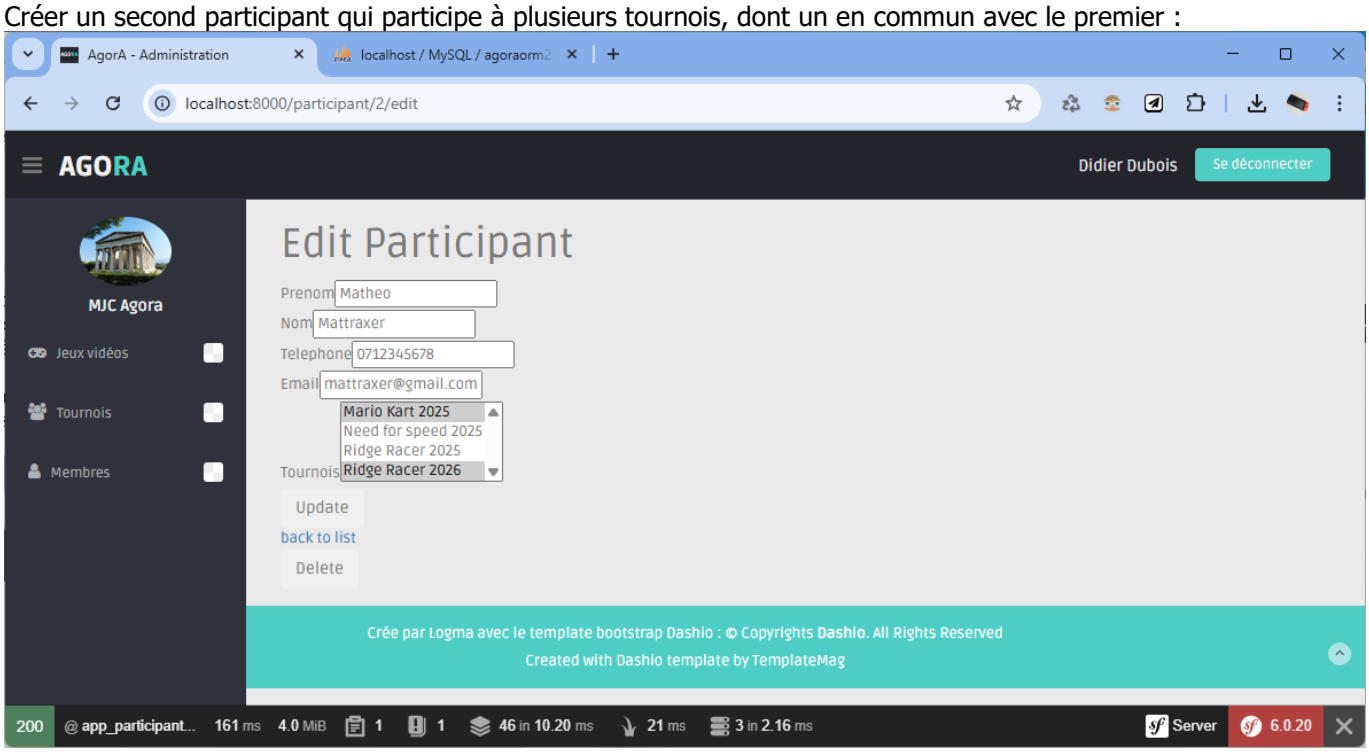
Créer plusieurs tournois :



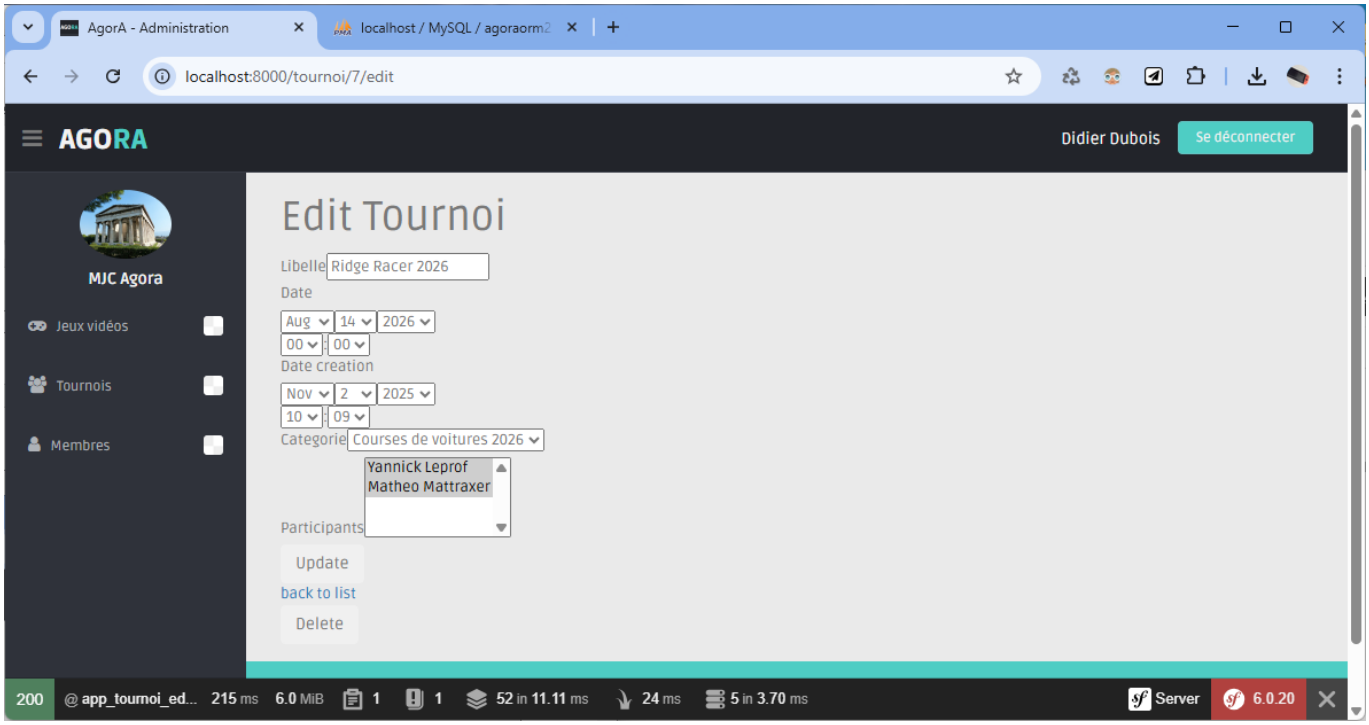
Créer un premier participant qui participe à plusieurs tournois :



Créer un second participant qui participe à plusieurs tournois, dont un en commun avec le premier :

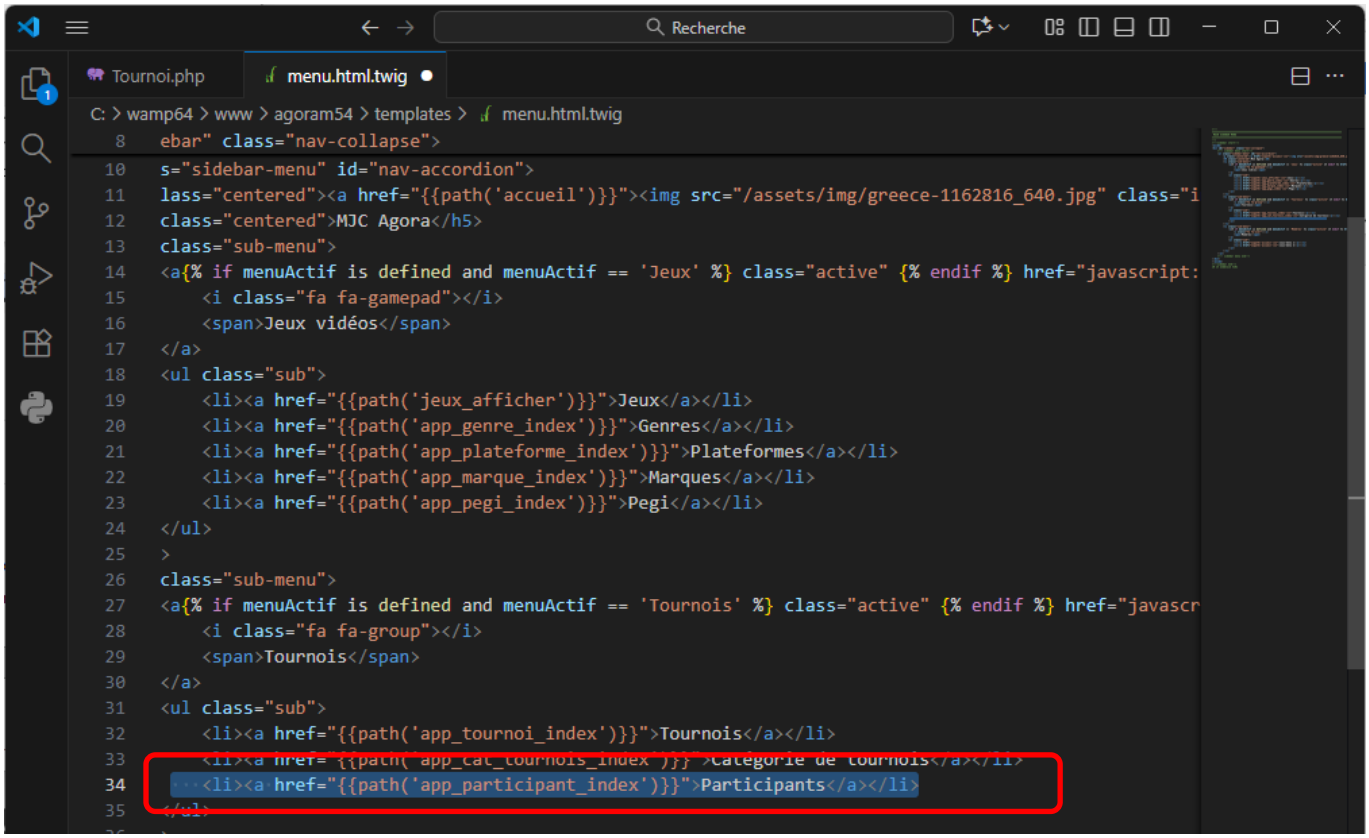


Puis éditer le tournoi en commun avec les 2 joueurs et constater qu'ils y sont bien inscrits :



2.4.5 Ajout du nouveau formulaire au menu

Il faut désormais ajouter le nouveau formulaire Participant au menu. Pour cela, il faut éditer templates/menu.html.twig :

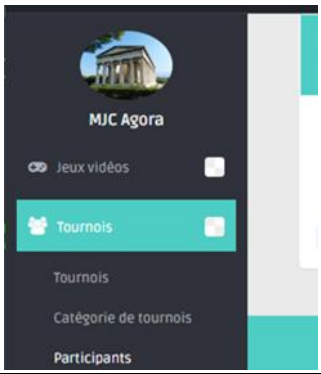


Il faut maintenir le menu « Tournois » ouvert lorsque l'on ouvre un de ce formulaires CRUD, pour cela il faut ajouter « 'menuActif' => 'Tournois', » à la fin des appels des vues du contrôleurs ParticipantController.php.

```php
<?php

namespace App\Controller;

use App\Entity\Participant;
use App\Form\ParticipantType;
use App\Repository\ParticipantRepository;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

#[Route('/participant')]
class ParticipantController extends AbstractController
{
    #[Route('/', name: 'app_participant_index', methods: ['GET'])]
    public function index(ParticipantRepository $participantRepository): Response
    {
        return $this->render('participant/index.html.twig', [
            'participants' => $participantRepository->findAll(), 'menuActif' => 'Tournois',
        ]);
    }
```

Ceci car le test dans le template menu.html.twig qui maintient le menu Tournois ouvert vérifie si une variable « menuActif » vaut « Tournois » :

```twig
<!--sidebar start-->
<aside>
<div id="sidebar" class="nav-collapse">
    <!-- sidebar menu start-->
    <ul class="sidebar-menu" id="nav-accordion">
        <p class="centered"><a href="{{path('accueil')}}"><img src="/assets/img/greece-11628
        <h5 class="centered">MJC Agora</h5>
        <li class="sub-menu">
            <a{% if menuActif is defined and menuActif == 'Jeux' %} class="active" {% endif
                <i class="fa fa-gamepad"></i>
                <span>Jeux vidéos</span>
            </a>
            <ul class="sub">
                <li><a href="{{path('jeux_afficher')}}">Jeux</a></li>
                <li><a href="{{path('app_genre_index')}}">Genres</a></li>
                <li><a href="{{path('app_plateforme_index')}}">Plateformes</a></li>
                <li><a href="{{path('app_marque_index')}}">Marques</a></li>
                <li><a href="{{path('app_pegi_index')}}">Pegi</a></li>
            </ul>
        </li>
        <li class="sub-menu">
            <a{% if menuActif is defined and menuActif == 'Tournois' %} class="active" {% en
                <i class="fa fa-group"></i>
                <span>Tournois</span>
            </a>
```



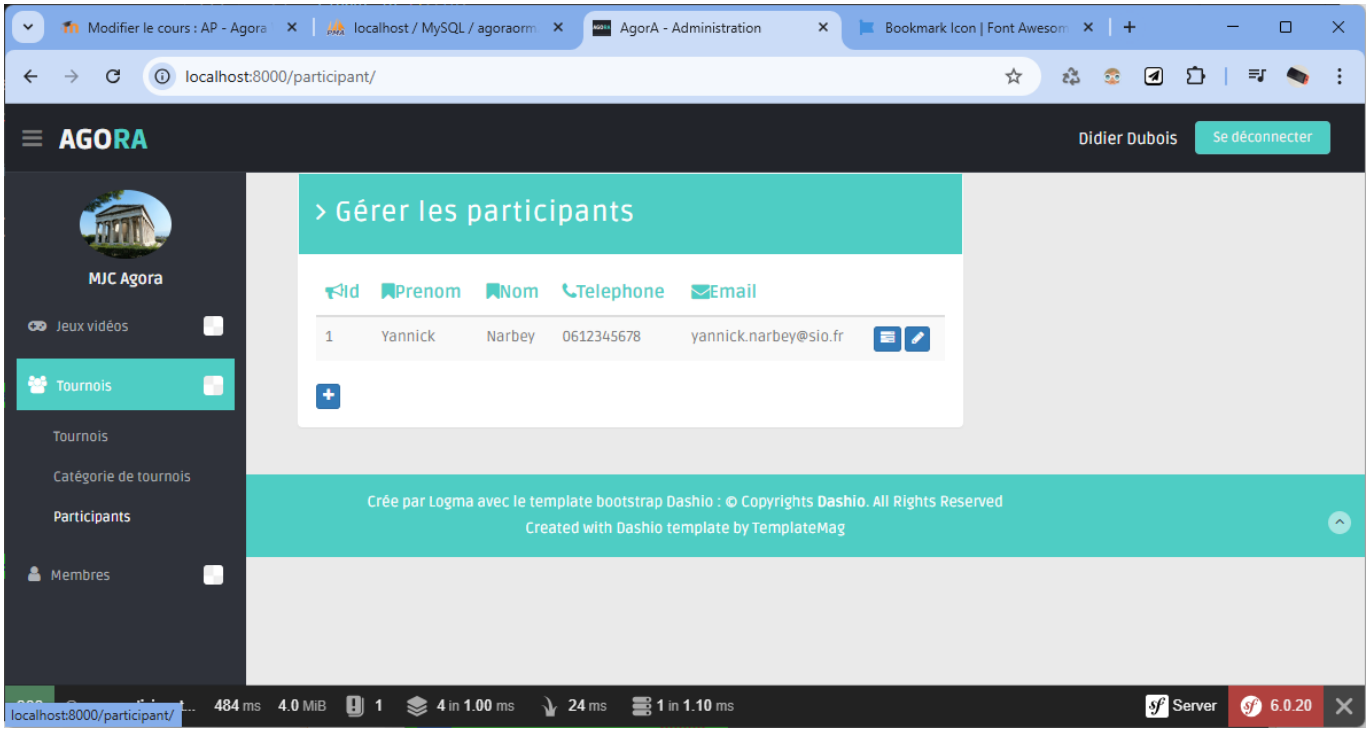
2.4.6 Charter et franciser le nouveau formulaire

Modifier le fichier templates/participant/index.html.twig pour charter et franciser le CRUD pour gérer les participants :

Fichier agora54/templates/participant/index.html.twig

```
{% extends 'base.html.twig' %}

{% block body %}
<!-- page start-->
<div class="col-sm-8">
    <section class="panel">
        <div class="chat-room-head">
            <h3><i class="fa fa-angle-right"></i> Gérer les participants</h3>
        </div>
        <div class="panel-body">
    <table class="table table-striped table-advance table-hover">
        <thead>
            <tr class="tableau-entete">
                <th><i class="fa fa-bullhorn"></i>Id</th>
                <th><i class="fa fa-bookmark"></i>Prenom</th>
                <th><i class="fa fa-bookmark"></i>Nom</th>
                <th><i class="fa fa-phone"></i>Telephone</th>
                <th><i class="fa fa-envelope"></i>Email</th>
                <th></th>
            </tr>
        </thead>
        <tbody>
        {% for participant in participants %}
            <tr>
                <td>{{ participant.id }}</td>
                <td>{{ participant.prenom }}</td>
                <td>{{ participant.nom }}</td>
                <td>{{ participant.telephone }}</td>
                <td>{{ participant.email }}</td>
                <td>
                    <a href="{{ path('app_participant_show', {'id':
participant.id}) }}"><button class="btn btn-primary btn-xs"><i class="fa fa-
tasks"></i></button></a>
                    <a href="{{ path('app_participant_edit', {'id':
participant.id}) }}"><button class="btn btn-primary btn-xs"><i class="fa fa-
pencil"></i></button></a>
                </td>
            </tr>
        {% else %}
            <tr>
                <td colspan="6">aucun participant enregistré en base de données</td>
            </tr>
        {% endfor %}
       </tbody>
    </table>

    <a href="{{ path('app_participant_new') }}"><button class="btn btn-primary
btn-xs"><i class="fa fa-plus"></i></button></a>
```

```
            </div><!-- fin div panel-body-->
        </section><!-- fin section tournois-->
    </div><!--fin div col-sm-8-->
{% endblock %}
```



2.4.7 Améliorations fonctionnelles

L'application est pleinement fonctionnelle, mais on peut ajouter quelques améliorations pour faciliter la gestion.

Par exemple, ajouter la liste des tournois auquel chaque participant est inscrit dans une colonne supplémentaire dans le CRUD participants :

Fichier agora54/templates/participant/index.html.twig

```
...
            <tr class="tableau-entete">
                <th><i class="fa fa-bullhorn"></i>Id</th>
                <th><i class="fa fa-bookmark"></i>Prenom</th>
                <th><i class="fa fa-bookmark"></i>Nom</th>
                <th><i class="fa fa-phone"></i>Telephone</th>
                <th><i class="fa fa-envelope"></i>Email</th>
                <th><i class="fa fa-gamepad"></i>Tournois</th>
                <th></th>
            </tr>
        </thead>
        <tbody>
        {% for participant in participants %}
            <tr>
                <td>{{ participant.id }}</td>
                <td>{{ participant.prenom }}</td>
                <td>{{ participant.nom }}</td>
                <td>{{ participant.telephone }}</td>
                <td>{{ participant.email }}</td>
```

```
<td>
{% if participant.tournois is not empty %}
    {% for tournoi in participant.tournois %}
        <span class="badge bg-info text-dark">{{ tournoi.libelle }}</span>
    {% endfor %}
{% else %}
    <em>Aucun</em>
{% endif %}
</td>
...
```

Tester :

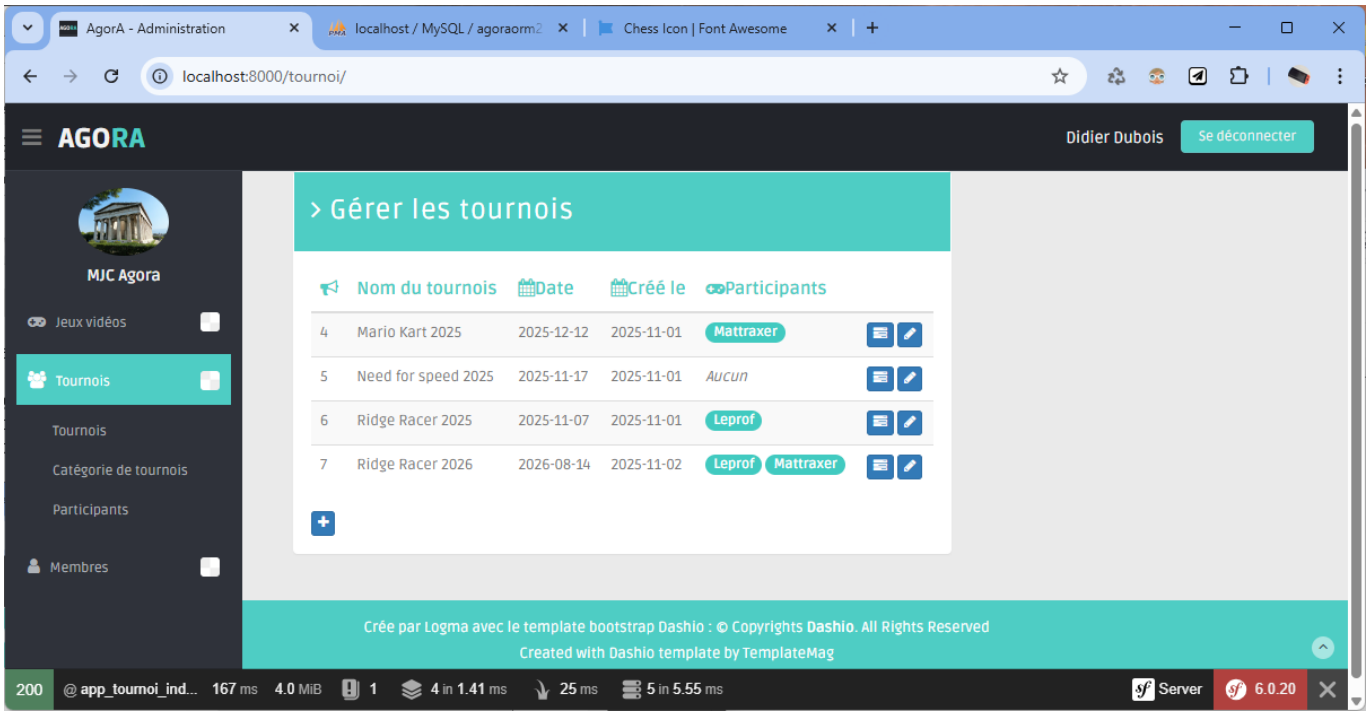
3. Conclusion

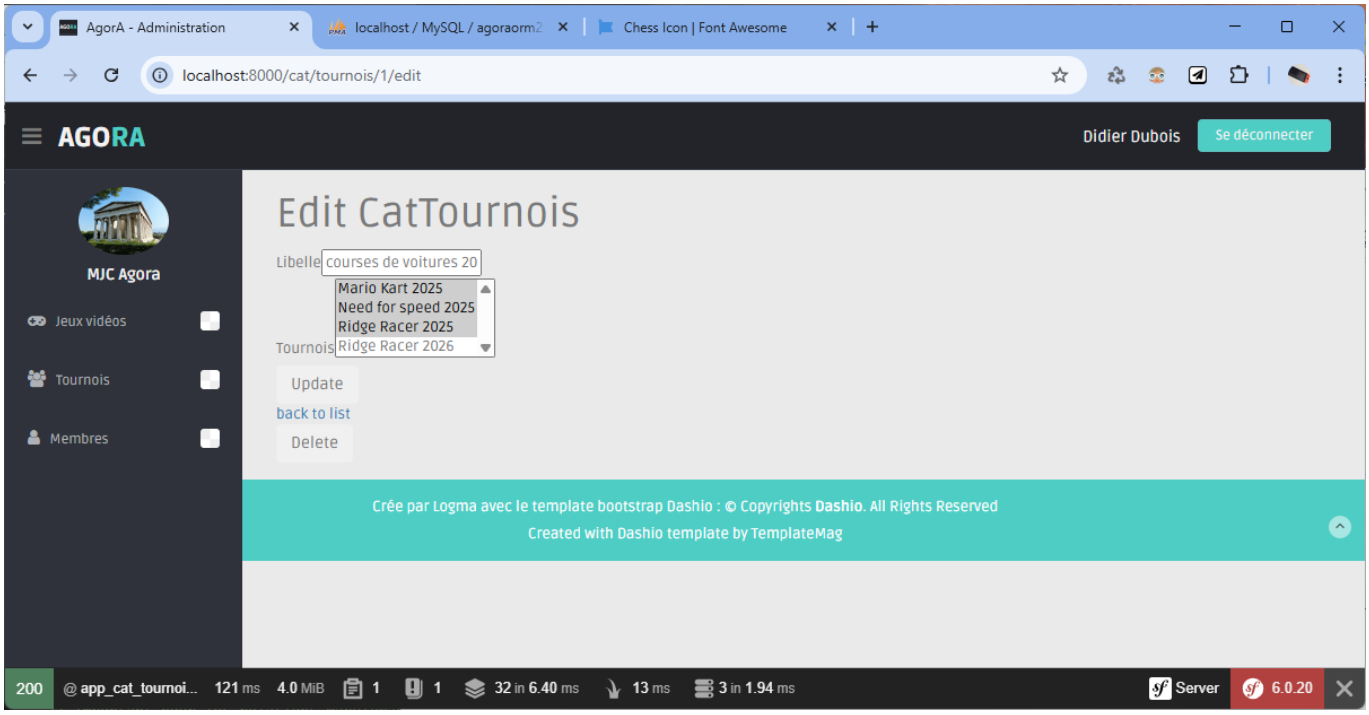
Vous avez désormais la méthodologie pour mettre en place une association many to many entre deux entités et construire les formulaires liés.

4. A vous de jouer

Mettez-vous en mode Scrum pour vous répartir l'amélioration les formulaires CRUD suivants :

- afficher la liste des participants sur le formulaire de gestion des tournois. (*)
- afficher la liste des tournois sur le formulaire de gestion des Catégories de Tournois. (*)
- ajouter la sélection des tournois dans l'écran d'édition ou de modification d'une catégorie de tournois (**)
- afficher la liste des jeux sur le formulaire de gestion des Plateformes. (*)
- afficher la liste des jeux sur le formulaire de gestion des Genres. (*)




AgorA - Administration
localhost / MySQL / agoraorm2
Chess Icon | Font Awesome
localhost:8000/cat/tournois/1/edit
AGORA
Didier Dubois
Se déconnecter
MJC Agora
Jeux vidéos
Tournois
Membres
Edit CatTournois
Libelle
courses de voitures 20
Mario Kart 2025
Need for speed 2025
Ridge Racer 2025
Tournois
Ridge Racer 2026
Update
back to list
Delete
Crée par Logma avec le template bootstrap Dashio : © Copyrights Dashio. All Rights Reserved
Created with Dashio template by TemplateMag
200
@ app_cat_tournoi...
121 ms
4.0 MiB
1
1
32 in 6.40 ms
13 ms
3 in 1.94 ms
Server
6.0.20